

Atividade Prática: Representação do Mundo de um Jogo Simples

IA para Jogos I - Solução desenvolvida em Python (pygame)

Activity: Representation of a Simple Game World

Course: Artificial Intelligence for Games I — Representation of the World | Professor: Charles Madeira Theme: Implementation of a game level with 9 areas on a regular grid, collectible items, NPCs controlled by simple automation and a viewport independent of the grid.

1. Testable Solution Link

The chosen publication option is to keep the project local (no automatic hosting). Therefore, below are the two ways to run and evaluate the solution.

1.1. Running locally (immediate)

```
pip install -r requirements.txt    # installs pygame
python main.py                    # starts the game
```

Tests (model without graphics):

```
python -m game.tests              # model unit tests
python -m game.simulation 20      # balance analysis (20 matches per scenario)
```

1.2. Link placeholder for online testing

To publish the game to be playable in the browser (nice to present in class), the recommended path is GitHub Pages + pygbag (a code that packages the pygame app as WebAssembly). After creating a repository:

```
pip install pygbag
pygbag main.py                # generates the /build folder
gh repo create <username>/simple-world-iaj --public --source=. --push
```

Then enable Pages on the repository (branch main, folder /build) — the game will be available at the link:

<https://ShinigameBR.github.io/python-simple-world-iaj/>

2. Statement Summary

Implement a game level in which the scenario is formed by 9 areas arranged in a grid containing collectible items and enemies (NPCs). The player must be able to control the displacement of their character, and NPCs must be controlled by a simple automation technique (e.g., pursue the player in a straight line). The player's viewing window (viewport) must be independent of the grid, with only nearby areas remaining active and the NPCs contained in them being updated.

The level's rules (slide 66–69):

- An area adjacent to the current one only becomes active when the character reaches "close" to its border;
- There will be at most 4 active areas;
- Only NPCs from active areas are updated — they move toward the player and, while overlapping, damage the player's health;
- A player can collect: first aid (health) — restores health — and ammunition (TNT) — deals damage to surrounding enemies;
- Any character (NPC or player) with non-positive health dies;

- Level objective: survive for a certain time;
- Initial state configurable (file/JSON or random) and balance testing in different scenarios (dozens/hundreds/thousands of enemies, small/medium/large areas, activation distance, damage, time...).

3. Design Choices

3.1. World representation: regular mesh (grid) of areas

The world was modeled as a regular grid of Areas (`game/world.py`), as recommended in the lesson for the "positioning of a character / nearby region" scale. Each Area (cell) defines an axis-aligned rectangle (limits, center) and concentrates its own NPCs and items. This is the key abstraction so that, at each iteration, it is only necessary to look at which cells are active and update their contents, instead of traversing all world entities.

This structure can also be seen as a navigation graph: each area is a node, and areas sharing a border are connected.

3.2. Position fixed in the cell center (cell-based abstraction)

Entities use absolute world coordinates (x, y), but each entity belongs to exactly one area, which is obtained by the Euclidean division of its coordinates by the area size (`World.cell_of`). This is the simplest form of "positioning within the shape": the cell (area) is calculated from the position, and collision tests are cheap. Items and NPCs belong to the cell's list (`Area.npcs`, `Area.items`), so buckets work as spatial hash.

Entity buckets are dynamic: when an NPC crosses a cell border while chasing the player, `World._migrate_npcs` re-registers it in the destination area's list (spatial-hash update). Without this, an enemy that physically entered an active area would remain registered in the old area — and would "disappear" whenever that old area was deactivated, even though it was standing inside the loading ring.

3.3. Active areas: loading ring (limited perception scope)

The dynamic reconfiguration of active areas lives in `World.update_active_areas`:

```
1 1
2 2
3 3
4 4
```

With the default activation distance ($1.6 \times \text{area_size}$), the ring around the player is always full: the current area + up to 3 nearest adjacent areas are active even when the player is at the center of an area. This prevents enemies from "vanishing" when the player moves to the middle of an area or crosses into another one — the areas around the player are continuously loaded, while far areas are frozen (their NPCs stop being simulated). With a short activation distance (e.g., 50–100px), the strict rule of the activity emerges: a neighboring area only activates when the character approaches its border.

This implements the concept of hierarchical abstraction / selective update and scales the simulation to worlds with dozens, hundreds or thousands of NPCs: only up to 4 areas have their NPCs updated per iteration.

3.4. Viewport independent of the mesh

Viewport (`game/camera.py`) is a rectangular window defined freely on the world ($x, y, \text{width}, \text{height}$), completely dissociated from grid cells: it can straddle the boundaries of several areas at once and does not depend on cell coordinates. Only entities inside the visible rectangle are drawn (culling). The camera follows the player smoothly (lerp) and is clamped to the world bounds.

The game opens maximized in windowed mode (fills the work area, excluding the taskbar). The camera also has a zoom factor (`viewport_zoom`, default 1.75): world coordinates are magnified on screen, so the viewport shows a closer, smaller region of the world (~1.5 areas wide) while entities/items are drawn proportionally smaller (radii in `config.json` and scaled icons). Zooming is purely visual — positions, collision and the area discretization keep working in world coordinates.

The perception scope (areas active for the simulation) and the visual window are two independent concepts — as highlighted in the lesson: position absolute for the AI vs. relative position to the viewport for visualization.

3.5. NPC AI: straight-line pursuit

`NPC.chase(target, dt)` computes the direction vector to the player, normalizes it and advances at constant speed. This is the simple automation requested ("pursue in a straight line"). Damage while overlapping: `contact_damage_to` checks overlap (`distance < radii sum`) and applies damage per second (`contact_damage * dt`). NPCs with `health <= 0` are removed in `_cleanup`.

3.6. Items and effects

- First aid: on collision with the player, `Player.heal` restores the configured amount (bounded by max health).
- Ammunition (TNT): rendered as a TNT crate with fuse (`_draw_tnt_icon`); on collision, applies area damage (`ammo_radius / ammo_damage`) to all NPCs within the radius (regardless of active area — because proximity is, physically, guaranteed).
- Explosion feedback (area damage): the model emits an event per hit (`World.explosions`); the renderer (`Effects`) draws a low-opacity filled flash that expands up to the damage radius plus a sharp expanding outline ring — making the area of effect visible to the player without blocking the scene.

3.7. Discrete-time simulation

The simulation advances with a fixed `dt` (1/120 cap in the game loop, 1/60 in the headless mode), characterizing a discrete simulation with automatically parallel (pseudo-parallel) entities — the lesson's first model. The balance test uses the same `World.update` so that the results reflect exactly the delivered mechanic.

3.8. Configuration and random generation

The level is generated randomly from `config.json` (JSON is a serialization of the initial state, permitted by the activity): sizes, NPC ranges per area, speed, damage, activation distance, survival time, seed. The seed guarantees reproducibility of the experiments.

3.9. Headless simulation for balance

`game/simulation.py` runs the world model without pygame under a simple, deterministic player policy (greedy: seek first aid when hurt; otherwise seek ammunition). It evaluates 12 scenarios × N seeds and produces the balance report. This satisfies the item "Test the balance in different scenarios" with a reproducible experiment instead of manual play.

4. Architecture

<code>main.py</code>	– pygame entry point: loop, input, states, HUD, rendering
<code>config.json</code>	– initial level state / parameters (JSON)
<code>game/</code>	
<code>entities.py</code>	– Player, NPC, Item (movement, health, effects)
<code>world.py</code>	– Area and World (grid, active areas, selective update)
<code>camera.py</code>	– Viewport (independent visualization window)
<code>simulation.py</code>	– headless balance analysis (<code>python -m game.simulation</code>)
<code>tests.py</code>	– model tests (<code>python -m game.tests</code>)
<code>smoke_test.py</code>	– screenshot/render smoke test (no window)
<code>docs/</code>	
<code>solucao.md</code>	– this document
<code>balance_report.md</code>	– generated balance report
<code>screenshots/</code>	– captured screenshots (menu, gameplay, pause)

Controls

Key	Function
WASD / Arrows	Move

Key	Function
P	Pause
R	Restart level
ENTER	Start (menu)
ESC / Q	Menu / Exit

5. Mapping to the Discipline Concepts

Lesson concept	Implementation
Regular grids as world abstraction	Grid of 9 Areas dividing the world into same-sized cells
Positioning within the cell	<code>cell_of()</code> computes the area from absolute coordinates
Entity bucket per cell	<code>Area.npcs / Area.items</code>
Position absolute × relative (viewport)	Viewport converts world → screen; AI uses absolute coordinates
Perception scope (limit nearby lookups)	<code>update_active_areas()</code> — loading ring, <code><= max_active(4)</code>
Selective update / hierarchy	Only NPCs from active areas run <code>chase/contact_damage</code>
Discrete simulation	fixed dt loop
Automated NPC (straight-line pursuit)	<code>NPC.chase</code>
Grid as graph	neighboring areas are implicitly connected
Trees/quadtrees (extension)	possible space-partitioning evolution (Section 7)

6. Balance Results

Generated by `python -m game.simulation 20` (see `docs/balance_report.md`). 20 matches per scenario; greedy player policy; survival time 90s unless noted.

Scenario	Enemies	Survival	Avg HP	Kills	Death (s)	Avg active areas
1. Baseline (3x3, 600px)	600px x9 (4–9/area)	50%	45.8	52	75.7	4.0
2. Dozens of enemies	600px x9 (3–5/area)	70%	66.3	33	80.3	4.0
3. Hundreds of enemies	600px x9 (40–60/area)	0%	0.0	46	3.4	4.0
4. Thousands of enemies	600px x9 (300–500/area)	0%	0.0	28	0.5	4.0
5. Small areas (300px)	300px x9 (4–9/area)	100%	98.5	59	–	4.0
6. Large areas (900px)	900px x9 (4–9/area)	30%	21.0	35	57.8	3.95
7. Short activation (100px)	600px x9 (4–9/area)	100%	95.4	53	–	1.23
8. Long activation (500px)	600px x9 (4–9/area)	65%	60.9	52	72.9	3.32

Scenario	Enemies	Survival	Avg HP	Kills	Death (s)	Avg active areas
9. Fast NPC (speed 220)	600px x9 (4–9/area)	15%	15.0	21	5.6	4.0
10. Lethal contact (60/s)	600px x9 (4–9/area)	25%	23.9	32	22.5	4.0
11. Survive 30s	600px x9 (4–9/area)	100%	84.8	52	–	4.0
12. Survive 180s	600px x9 (4–9/area)	40%	37.2	52	80.9	4.0

Analysis

- Loading ring (default): with the default activation distance, the 4 surrounding areas stay active most of the time (avg 4.0). Enemies never "vanish" near the player; far areas remain frozen (not simulated).
- Entity migration: since NPCs are re-bucketed when they cross area borders (`_migrate_npcs`), enemies that enter an active area keep being updated and attacking — encounters near the ring are persistent instead of "disappearing".
- Scaling: dozens → hundreds → thousands multiply the crowd in the loaded ring; in scenarios 3–4 the greedy policy dies before collecting enough TNT (automatic massacre). Thousands of distant enemies only "exist" but are not simulated until their area enters the ring — which proves the selective update's objective.
- Area size: small areas (~4 active on average) increase confrontations with identical density; large areas dilute the NPCs but, since the ring covers a larger physical area (up to 3×3), more NPCs are active at once — harder (30%).
- Activation distance: with a short radius the strict "near the border" rule emerges (avg 1.23 active areas, 100% survival); long activation approaches the ring mode (65%).
- NPC speed and contact damage are the dominant difficulty factors (15% and 25% survival) — they are the recommended parameters to tune difficulty.
- With a passive player the default map is challenging (50% survival): the greedy policy clears its ring of enemies with TNT, but NPCs that chase across the border keep coming. A human player who moves and kites has more margin.

Tuned default: with the human player in mind, `config.json` was left at speed 260 / NPC 150, damage 25/s, health pickups +40, TNT radius 220 and 90s of survival — a balanced base for manual play.

7. Extensions / Bonus Implemented as Possible

- Dynamic load/release (bonus): the active-area mechanism already deactivates areas far from the player, freezing their NPCs. On worlds much larger than 3×3, the areas could also have their entity lists unloaded from memory, re-instantiated only on activation (`World.update_active_areas` is the natural hook).
- Quadtree: replacing the fixed grid with a Quadtree (`getObjects(x, y)` as in the lesson) is a direct evolution, keeping the entity-per-region bucket concept.
- Navigation graph / BFS pathfinding: since the grid is a graph, chase could later use neighborhood BFS to avoid obstacles.
- Difficulty curve: gradual activation distance and speeds via the same `config.json` fields.

Fallback: the balance report, unit tests and screenshots can be regenerated with the commands in Section 1.1.

Apêndice - Screenshots da solução



Tela inicial (menu)



Jogando: viewport, áreas ativas e entidades



Tela pausada